Zero to Hero Study Handbook: Voice AI Study Coach

This handbook is built from static analysis of the repository files only (no runtime execution in this session). It is designed for a new learner who wants to understand the architecture, code logic, and design patterns end-to-end.

Module 1: Foundations & Architecture

1) What this project does

`voice-ai-study-coach` is a notebook-first, local voice tutoring system that:

1. Takes a student question as text or audio.
2. Retrieves relevant notes from local knowledge documents.
3. Generates a tutor explanation and a follow-up quiz question using local Ollama models.
4. Synthesizes tutor/quiz outputs into `.wav` files.
5. Records telemetry spans and evaluation artifacts for analysis.

Primary runtime references:

- `src/voice_study_coach/cli.py`
- `src/voice_study_coach/pipeline.py`
- `src/voice_study_coach/orchestration/session.py`

Main use cases in this repo:

1. End-to-end demo sessions (`run-demo`, `run-all`).
2. Evaluation on curated tasks (`evaluate`).
3. Telemetry aggregation and markdown reporting (`summarize-telemetry`, `render_report`).
4. Tutorial learning through notebooks in `notebooks/`.

2) Core paradigms and patterns used here

1. Object-oriented composition (definition: build behavior by composing focused classes).
   - Used in `VoiceStudySession`, which composes `TutorAgent`, `QuizAgent`, `SidecarTranscriber`, `ProceduralVoiceSynthesizer`, retrieval docs, and telemetry tracer.
2. Async orchestration (definition: non-blocking control flow using `async`/`await`).
   - `run_demo`, `run_evaluation`, `run_all`, and model calls in `AsyncOllamaGateway.chat` are async.
3. Dependency injection (definition: pass dependencies into constructors instead of creating hidden globals).
   - `VoiceStudySession.__init__` receives all major components explicitly.

4. Adapter/Gateway pattern (definition: wrap external systems behind small interfaces).
    - `AsyncOllamaGateway` wraps Ollama API calls.
    - `SidecarTranscriber` adapts audio transcription to sidecar `.txt`.
    - `ProceduralVoiceSynthesizer` adapts text to WAV generation.
5. Retrieval-augmented generation (RAG-lite) with lexical scoring (definition: retrieve context first, then generate response).
    - `tools/knowledge_base.py` computes overlap-based scores.
    - `TutorAgent.explain` and `QuizAgent.generate_quiz` consume retrieved docs.
6. Fallback-first reliability (definition: return deterministic fallback outputs when model calls fail/timeout).
    - Both agents catch exceptions and return fallback `ChatResult` values with `fallback_used=True`.
7. Typed data contracts (definition: explicit schemas for IO and artifacts).
    - Pydantic models in `schemas.py` define `EvalTask`, `SessionResult`, `EvalPrediction`, `EvalSummary`, `TraceSpanRecord`, etc.

3) Architecture and component interactions

Key components:

1. Configuration: `Settings` in `config.py` (env-driven paths/models/timeouts/audio knobs).
2. CLI: Typer app in `cli.py` exposing user commands.
3. Pipeline: `run_demo`, `run_evaluation`, `run_all` in `pipeline.py`.
4. Session orchestrator: `VoiceStudySession.run`.
5. Agents: `TutorAgent`, `QuizAgent`.
6. Retrieval: `load_docs`, `retrieve`.
7. Audio I/O: `SidecarTranscriber`, `ProceduralVoiceSynthesizer`.
8. Telemetry: `JsonlTelemetryTracer`, `summarize_traces`.
9. Reporting: CSV/JSON writers and Jinja markdown report renderer.

Main flow diagram:

```
User / CLI Command
    |
    v
voice-study-coach (Typer app in cli.py)
    |
    v
get_settings() -> Settings (paths/models/runtime knobs)
    |
    v
pipeline.py (run_demo / run_evaluation / run_all)
    |
    v
```

```
_build_session(...)
    |
    v
VoiceStudySession.run(trace_id, question_text | question_audio_path)
    |
    +--> [audio input only] SidecarTranscriber.transcribe(<audio>.wav -> <audio>.txt)
    |
    +--> retrieve(query, docs, top_k)
    |
    +--> TutorAgent.baseline_answer(question)
    |
    +--> TutorAgent.explain(question, retrieved)
    |
    +--> QuizAgent.generate_quiz(question, tutor_answer, retrieved)
    |
    +--> ProceduralVoiceSynthesizer.synthesize(tutor_text) -> *_tutor.wav
    |
    +--> ProceduralVoiceSynthesizer.synthesize(quiz_text)  -> *_quiz.wav
    |
    v
SessionResult (structured payload)
    |
    +--> Evaluation: evaluate(...) -> EvalPrediction rows + EvalSummary
    +--> Telemetry: traces.jsonl + summary.json
    +--> Reporting: predictions.csv + summary.json + report.md + run_summary.json
```

Module 2: Repository Map

Focus first on the files below; they define almost all runtime behavior.

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`README.md`	Project overview, setup commands, CLI examples, artifact expectations	N/A	Model names, notebook order, command list
`pyproject.toml`	Packaging, dependencies, CLI entrypoint, lint/test config	`[project.scripts]` `voice-study-coach = "voice_study_coach.cli:app"`	Python range `>=3.12.10,<3.13`, dependencies (`ollama`, `pydantic-settings`, `typer`, etc.)

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`.env.example`	Environment variable contract	N/A	`OLLAMA_HOST`, `TUTOR_MODEL`, `QUIZ_MODEL`, path variables, generation/audio knobs
`scripts/run_pipeline.py`	Thin convenience wrapper around `run_all`	`_main()`	Uses `get_settings()`, `configure_logging()`
`scripts/execute_notebooks.py`	Sequential notebook execution helper	`NOTEBOOKS`, `execute()`, `main()`	Notebook timeout `1800`, kernel `python3`
`scripts/generate_notebooks.py`	Programmatic notebook authoring	`_write()`, `main()`	Notebook file list and tutorial cell content
`src/voice_study_coach/config.py`	Settings model and path resolution	`Settings`, `resolve()`, `ensure_dirs()`, `get_settings()`	`retrieval_top_k`, `generation_timeout_seconds`, `audio_sample_rate`, output file properties
`src/voice_study_coach/cli.py`	CLI entry point and command handlers	`app`, `check_backends_cmd`, `run_demo_cmd`, `evaluate_cmd`, `run_all_cmd`, `summarize_telemetry_cmd`	callback initializes settings + logging
`src/voice_study_coach/pipeline.py`	Main orchestration for demos/evals/full run	`_build_session`, `_prepare_demo_audio`, `run_demo`, `run_evaluation`, `run_all`	Demo trace IDs, artifact write paths, trace reset behavior
`src/voice_study_coach/orchestration/session.py`	Single-session runtime flow	`VoiceStudySession.run`	Span names (`transcribe_audio`, `retrieve_context`, `tutor_answer`, etc.), `top_k`
`src/voice_study_coach/agents/tutor.py`	Tutor response generation + fallback	`TutorAgent.baseline_answer`, `TutorAgent.explain`	Uses `settings.tutor_model`, generation limits and timeout

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
src/voice_study_coach/agents/quiz.py	Quiz generation + fallback	QuizAgent.generate_quiz	Uses settings.quiz_model, token/time caps
src/voice_study_coach/ollama_client.py	Async Ollama API wrapper	AsyncOllamaGateway.chat, ensure_required_models	options.temperature, options.num_predict, timeout wrapper
src/voice_study_coach/knowledge/base.py	Knowledge loading and lexical retrieval	load_base, retrieve, _tokenize, _extract_title	SUPPORTED_SUFFIXES = {".md", ".txt"}
src/voice_study_coach/audio/transcription.py	Audio transcription adapter	SidecarTranscriber.transcribe	Expects <audio>.txt; returns fallback transcript if missing
src/voice_study_coach/audio/synthesis.py	Piper/fallback text-to-WAV synthesis	PiperEspeakVoiceSynthesizer.synthesize, _word_tone	sample_rate, amplitude, word_seconds, pause_seconds
src/voice_study_coach/schemas.py	Typed contracts for all major payloads	KnowledgeDoc, RetrievedDoc, ChatResult, SessionResult, EvalTask, EvalPrediction, EvalSummary, TraceSpanRecord	Strict extra="forbid" across models
src/voice_study_coach/eval/evaluator.py	Eval task metrics and aggregation	load_tasks, keyword_recall, evaluate	trace_id = f"eval-{idx:03d}-{task.task_id}"
src/voice_study_coach/telemetry/tracer.py	Span logging and aggregate telemetry stats	JsonlTelemetryTracer.span, summarize_traces	JSONL span fields + per-span p50/p95/max stats
src/voice_study_coach/reporting/report.py	CSV/JSON writers and markdown report generation	save_predictions, save_summary, save_demo_runs, render_report	Jinja _REPORT_TEMPLATE sections and metrics placeholders
data/knowledge/*.md	Source corpus for retrieval	N/A	Content used for tutor/quiz context and eval source matching

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`data/eval/tasks.json`	Evaluation task set	N/A (loaded into `EvalTask`)	`task_id`, `question`, `reference_answer`, `expected_source`, `required_keywords`
`tests/*.py`	Behavior checks for retrieval, telemetry, eval metric, audio adapters	`test_retrieve_top_k`, `test_summarize_traces`, etc.	Captures expected behavior of key units

Module 3: Core Execution Flows

Flow A: CLI entry and command dispatch

Entry declaration in `pyproject.toml`:

```
[project.scripts]
voice-study-coach = "voice_study_coach.cli:app"
```

Runtime behavior in `cli.py`:

1. `callback()` runs before each command.
2. It calls `get_settings()`, `configure_logging()`, and `settings.ensure_dirs()`.
3. Individual commands call async pipeline functions via `_run_async(...)`.

Important command mapping:

1. `check-backends` -> local model inventory (`AsyncOllamaGateway.list_model_names`).
2. `run-demo` -> `run_demo(settings)`.
3. `evaluate` -> `run_evaluation(settings, reset_traces=False)`.
4. `run-all` -> `run_all(settings)`.
5. `summarize-telemetry` -> `summarize_traces(...)`.

Flow B: Full end-to-end run (`run_all`)

Code path: `cli.py:run_all_cmd` -> `pipeline.py:run_all`.

Step-by-step:

1. If `settings.traces_file` exists, it is deleted.
2. `run_demo(settings)` runs 3 predefined sessions.
3. `run_evaluation(settings, reset_traces=False)` runs all tasks from `data/eval/tasks.json`.
4. Payload is assembled with model names, demo counts, evaluation payload, and output path.

5. Payload is written to `settings.run_summary_file` (`artifacts/run_summary.json` by default).

Short source fragment (from `pipeline.py`):

```
payload = {
    "tutor_model": settings.tutor_model,
    "quiz_model": settings.quiz_model,
    "demo_runs_path": settings.demo_runs_file.as_posix(),
    "demo_count": len(demo_runs),
    "evaluation": eval_payload,
    "run_summary_path": settings.run_summary_file.as_posix(),
}
```

Flow C: Single session orchestration (`VoiceStudySession.run`)

Code path: `pipeline._build_session(...)` constructs `VoiceStudySession`; then session `run(...)` executes one turn.

Method signature:

```
async def run(
    self,
    trace_id: str,
    *,
    question_text: str | None = None,
    question_audio_path: Path | None = None,
) -> SessionResult:
```

Execution order:

1. Input mode resolution:
   - If `question_audio_path` exists, transcribe via `SidecarTranscriber.transcribe`.
   - Else use `question_text` directly.
2. Retrieval:
   - `retrieve(query=question, docs=self._docs, top_k=self._top_k)`.
   - Retrieval score = lexical overlap ratio.
3. Tutor baseline span:
   - `await self._tutor.baseline_answer(question)` (for tracing baseline phase).
4. Tutor contextual response:
   - `await self._tutor.explain(question=question, retrieved=hits)`.
5. Quiz generation:
   - `await self._quiz.generate_quiz(question=question, tutor_answer=tutor_result.text, retrieved=hits)`.
6. Audio synthesis:
   - Tutor text -> `{trace_id}_tutor.wav`
   - Quiz text -> `{trace_id}_quiz.wav`

7. `SessionResult` returned with all text, retrieval metadata, fallback flags, audio paths, durations, and total latency.

Telemetry span names used by this flow:

- `transcribe_audio` (audio only)
- `retrieve_context`
- `baseline_answer`
- `tutor_answer`
- `quiz_generation`
- `synthesize_tutor_audio`
- `synthesize_quiz_audio`

Flow D: Evaluation loop (`run_evaluation` + `evaluate`)

Code path: `pipeline.py:run_evaluation` -> `eval/evaluator.py:evaluate`.

Step-by-step:

1. Optional trace reset (`reset_traces` flag).
2. Build session.
3. Load tasks via `load_tasks(settings.resolved_evaluation_file.as_posix())`.
4. For each task:
   - Build `trace_id` as `eval-<idx>-<task_id>`.
   - Run session with `question_text`.
   - Compute `keyword_recall` for baseline and tutor answers.
   - Compute `retrieval_hit` and `source_cited`.
   - Append `EvalPrediction`.
5. Aggregate `EvalSummary` means/rates.
6. Save artifacts:
   - CSV predictions (`save_predictions`)
   - JSON summary (`save_summary`)
   - Telemetry summary (`summarize_traces`)
   - Markdown report (`render_report`)

Important implementation detail:

- In `evaluate`, baseline text is currently fixed to `"I do not know from the question alone."` for baseline keyword scoring.

Flow E: Telemetry summarization and reporting

Telemetry write path:

1. Every span context manager call appends one `TraceSpanRecord` JSON line to `traces.jsonl`.
2. `summarize_traces` groups rows by `span_name`.
3. It computes count, error count, mean latency, p50, p95, and max.
4. Summary JSON is written to `artifacts/telemetry/summary.json`.

Report generation path:

1. `render_report(...)` renders Jinja template `_REPORT_TEMPLATE`.
2. Template sections: setup, metrics, telemetry table, per-task scores.
3. Markdown report saved at `artifacts/reports/voice_study_coach_report.md`.

Key input/output shapes (from real code)

`EvalTask` item shape (`data/eval/tasks.json`):

```
{
  "task_id": "q1",
  "question": "When should emergency rollback be triggered for API incidents?",
  "reference_answer": "Rollback is required when p95 latency exceeds 1200 ms for 10 consecut
  "expected_source": "incident_playbook.md",
  "required_keywords": ["1200", "10", "minutes", "rollback"]
}
```

`SessionResult` shape (`schemas.py` + `session.py` construction):

```
{
  "trace_id": "demo-audio-001",
  "input_mode": "audio",
  "question_text": "...",
  "retrieved": [
    {
      "source": "incident_playbook.md",
      "title": "Incident Playbook",
      "score": 0.6364,
      "text": "..."
    }
  ],
  "tutor_response_text": "...",
  "quiz_question_text": "...",
  "tutor_audio_path": "artifacts/audio/<trace>_tutor.wav",
  "quiz_audio_path": "artifacts/audio/<trace>_quiz.wav",
  "tutor_audio_seconds": 7.846,
  "quiz_audio_seconds": 1.774,
  "tutor_fallback_used": true,
  "quiz_fallback_used": true,
  "total_latency_ms": 7022.152
}
```

`EvalPrediction` CSV columns (`artifacts/evals/predictions.csv`):

1. `task_id`
2. `question`
3. `expected_source`
4. `baseline_answer`

5. tutor_answer
6. baseline_keyword_recall
7. tutor_keyword_recall
8. keyword_gain
9. retrieval_hit
10. source_cited
11. tutor_fallback_used
12. quiz_fallback_used
13. tutor_audio_seconds
14. total_latency_ms

Telemetry summary shape (`summarize_traces` output):


```
{
  "generated_at_utc": "2026-06-12T11:44:32+00:00",
  "trace_file": ".../artifacts/telemetry/traces.jsonl",
  "n_spans": 175,
  "n_unique_traces": 10,
  "by_span": [
    {
      "span_name": "tutor_answer",
      "count": 28,
      "error_count": 0,
      "latency_ms_mean": 3005.983,
      "latency_ms_p50": 3004.845,
      "latency_ms_p95": 3012.842,
      "latency_ms_max": 3021.719
    }
  ]
}
```


Module 4: Setup & Run Guide

1) Prerequisites

1. Linux/macOS shell environment.
2. Python `3.12.10` (project requires `>=3.12.10,<3.13`).
3. `uv` package manager.
4. Ollama running locally (default `http://127.0.0.1:11434`).

2) Clean-machine setup

```
git clone https://github.com/pypi-ahmad/voice-ai-study-coach.git
cd voice-ai-study-coach
uv python pin 3.12.10
uv sync --dev
cp .env.example .env
```

Pull required local models:

```
ollama pull phi3.5:3.8b
ollama pull functiongemma:270m
```

3) Environment configuration

All required keys are documented in `.env.example`.

Core backend keys:

1. `OLLAMA_HOST`
2. `TUTOR_MODEL`
3. `QUIZ_MODEL`

Generation and retrieval knobs:

1. `SEED`
2. `RETRIEVAL_TOP_K`
3. `GENERATION_TEMPERATURE`
4. `GENERATION_MAX_TOKENS`
5. `GENERATION_TIMEOUT_SECONDS`

Audio knobs:

1. `AUDIO_SAMPLE_RATE`
2. `AUDIO_AMPLITUDE`
3. `WORD_SECONDS`
4. `PAUSE_SECONDS`

Data/artifact paths:

1. `KNOWLEDGE_DIR`
2. `EVALUATION_FILE`
3. `DEMO_AUDIO_DIR`
4. `ARTIFACTS_DIR`
5. `AUDIO_DIR`
6. `EVAL_DIR`
7. `REPORT_DIR`
8. `RUNS_DIR`
9. `TELEMETRY_DIR`

4) Typical command sequences

Backend check:

```
uv run voice-study-coach check-backends
```

Run demo sessions only:

```
uv run voice-study-coach run-demo
```

Run evaluation only:

`uv run voice-study-coach evaluate`

Run complete pipeline:

`uv run voice-study-coach run-all`

Summarize telemetry from traces:

`uv run voice-study-coach summarize-telemetry`

Notebook tutorial execution:

`uv run python scripts/execute_notebooks.py`

5) Migration/seeding notes

1. No database migration framework exists in this repo.
2. No DB seed scripts exist.
3. Knowledge base content is file-based under `data/knowledge/`.
4. Evaluation task set is file-based under `data/eval/tasks.json`.
5. Demo audio files are generated by `_prepare_demo_audio(settings)` in `pipeline.py`.
6. Output directories are auto-created by `Settings.ensure_dirs()`.

Module 5: Study Plan & Practice Exercises

1) Ordered study plan for a new learner

1. Read `README.md` and `pyproject.toml`.
2. Read `src/voice_study_coach/config.py` to understand runtime knobs and paths.
3. Read `src/voice_study_coach/schemas.py` to learn the data contracts.
4. Read `src/voice_study_coach/cli.py` and `src/voice_study_coach/pipeline.py` for top-level control flow.
5. Read `src/voice_study_coach/orchestration/session.py` for one full interaction lifecycle.
6. Read `src/voice_study_coach/tools/knowledge_base.py` to understand retrieval mechanics.
7. Read `src/voice_study_coach/agents/tutor.py`, `agents/quiz.py`, `ollama_client.py` for model interaction and fallbacks.
8. Read `src/voice_study_coach/audio/*.py` for I/O adapters.
9. Read `src/voice_study_coach/eval/evaluator.py`, `telemetry/tracer.py`, and `reporting/reporting.py` for measurement/reporting.
10. Read `tests/` to confirm expected behavior with concrete examples.

2) Practice exercises (with solution outlines)

Exercise 1: Trace one audio question end-to-end.

- Task: Starting at `run_demo`, identify every function call that leads to `demo-audio-001_tutor.wav`.
- Solution outline: `run_demo` -> `_prepare_demo_audio` -> `_build_session` -> `session.run(question_audio_path=...)` -> `transcribe` -> `retrieve` -> `TutorAgent.explain` -> `QuizAgent.generate_quiz` -> `synthesize` to tutor WAV.

Exercise 2: Explain retrieval scoring mathematically.

- Task: From `knowledge_base.retrieve`, write the exact score formula and explain what tokens are compared.
- Solution outline: `score = len(q_terms & d_terms) / max(1, len(q_terms))`, where terms come from lowercased, punctuation-stripped whitespace tokens.

Exercise 3: Identify all fallback paths.

- Task: List every fallback output path in tutor, quiz, and transcription.
- Solution outline: `TutorAgent.baseline_answer` fallback text, `TutorAgent.explain` fallback summary or no-context message, `QuizAgent.generate_quiz` fallback question, `SidecarTranscriber.transcribe` fallback transcript when sidecar missing/empty.

Exercise 4: Add a new eval task correctly (conceptual exercise).

- Task: Describe the minimum JSON fields required to add `q7` to `data/eval/tasks.json`.
- Solution outline: Must include `task_id`, `question`, `reference_answer`, `expected_source`, `required_keywords` (list), matching `EvalTask` schema.

Exercise 5: Compare text-input and audio-input session branches.

- Task: Which spans run only for audio input?
- Solution outline: `transcribe_audio` runs only when `question_audio_path` is provided; all other spans run for both modes.

Exercise 6: Explain how output file paths are formed.

- Task: Find where `*_tutor.wav` and `*_quiz.wav` names are generated.
- Solution outline: `session.py`, `tutor_audio = audio_output_dir / f"{trace_id}_tutor.wav"` and similarly for quiz.

Exercise 7: Reconstruct evaluation output columns without opening CSV.

- Task: Use `EvalPrediction` schema to list expected CSV headers.
- Solution outline: Headers come from `row.model_dump()` key order in `save_predictions`; fields are the 14 columns listed in Module 3.

Exercise 8: Explain why telemetry summary includes p95.

- Task: Identify implementation lines that compute percentiles and explain why p95 is useful.

- Solution outline: `_safe_percentile(..., 0.95)` in `tracer.py`; p95 helps detect high-latency tail behavior not visible in mean/p50.

Exercise 9: Understand path resolution behavior.

- Task: If `EVAL_DIR` in `.env` is relative, where does it resolve?
- Solution outline: `Settings.resolve` prepends `project_root` for non-absolute paths.

Exercise 10: Inspect notebook architecture linkage.

- Task: Explain how `06_full_tutorial_voice_ai_study_coach.ipynb` mirrors package-level modules.
- Solution outline: Notebook imports config, gateway, retrieval, audio adapters, agents, session, telemetry, and pipeline functions in the same dependency order as runtime.

Verification Checklist

Use this checklist to validate your understanding:

1. Can you explain how `voice-study-coach run-all` traverses `cli.py` to `pipeline.py` to artifact files?
2. Can you describe the two input modes in `VoiceStudySession.run` and where they diverge?
3. Can you derive retrieval scores exactly from query/doc token overlap rules?
4. Can you list all fallback behaviors and when each one triggers?
5. Can you explain the purpose of each major schema in `schemas.py`?
6. Can you map telemetry span names to exact runtime stages?
7. Can you explain how evaluation metrics are computed from per-task rows?
8. Can you locate where markdown report content is templated and rendered?
9. Can you add a valid new task in `data/eval/tasks.json` without breaking schema expectations?
10. Can you describe which `.env` variables directly affect latency, output length, and retrieval depth?